

SQL Server — Lista de Exercícios

Complemento prático ao curso **SQL para Desenvolvedores** (desenvolvedor.io).
Um bloco de exercícios por módulo, todos sobre o mesmo banco de prática.

Como usar esta lista

- Assista à aula, faça as anotações no caderno e, em seguida, resolva os exercícios do módulo correspondente.
- Cada módulo vira um arquivo **.sql** no repositório (ex.: **03-hands-on-basico.sql**), com o número do exercício em comentário (**-- 3.4**) antes da solução.
- Não consulte soluções antes de tentar. Se travar, releia a anotação do caderno e a documentação oficial (learn.microsoft.com).
- Todo **UPDATE** ou **DELETE** começa como um **SELECT** com o mesmo **WHERE**. Confira as linhas afetadas antes de alterar.
- Escreva todos os scripts à mão. Ferramentas visuais (designer de tabelas, assistentes) não contam como solução.
- Marque o ☐ ao concluir. Um exercício só está concluído quando você consegue explicar a solução sem olhar.

Banco de prática: DistribuidoraDB

Todos os módulos usam o mesmo banco, que evolui ao longo do curso. O domínio é uma distribuidora que vende produtos a clientes, com pedidos registrados por vendedores. Os tipos de dados **não** estão definidos de propósito: escolhê-los faz parte do exercício 1.3.

Tabela	Colunas	Observações
Categorias	CategoriaId, Nome	Nome obrigatório.
Produtos	ProdutoId, Nome, CategoriaId, Preco, Estoque, Ativo, DataCadastro	CategoriaId referencia Categorias.
Clientes	ClienteId, Nome, Email, Cidade, UF, DataCadastro	Email único por cliente.
Vendedores	VendedorId, Nome, PercentualComissao	—
Pedidos	PedidoId, ClienteId, VendedorId, DataPedido, Status	VendedorId aceita NULL (venda direta). Status: Aberto, Faturado, Cancelado.
ItensPedido	PedidoId, ProdutoId, Quantidade, PrecoUnitario	PK composta (PedidoId, ProdutoId). PrecoUnitario guarda o preço do momento da venda.

Estrutura sugerida do repositório

```
sql-estudos/  
  README.md  
  lista-de-exercicios.pdf  
  00-setup.sql          -- criação do banco e tabelas  
  01-carga-inicial.sql -- inserts de dados de teste  
  03-hands-on-basico.sql  
  04-base-conhecimentos.sql  
  ...  
  procedures/          -- um arquivo por procedure
```

Módulo 1 — Iniciando no mundo SQL

Módulo teórico. Resolva no caderno, sem computador. O objetivo é projetar o banco antes de escrever qualquer script.

- ☐ **1.1** Liste as entidades e os atributos da DistribuidoraDB e defina a chave primária de cada tabela.
- ☐ **1.2** Desenhe o diagrama de relacionamentos, indicando quais são 1:N e quais são N:N. Explique por que a tabela **ItensPedido** precisa existir.
- ☐ **1.3** Escolha o tipo de dado de cada coluna e justifique. Obrigatório responder: por que **DECIMAL** e não **FLOAT** para valores monetários? Qual a diferença entre **VARCHAR** e **NVARCHAR** e quando usar cada um?
- ☐ **1.4** Para cada chave estrangeira, descreva o que deveria acontecer se o registro pai fosse excluído (bloquear, excluir em cascata ou deixar NULL) e por quê.
- ☐ **1.5** Responda: por que Email não é uma boa chave primária para Clientes, mesmo sendo único? Defina chave natural e chave substituta (surrogate key).
- ☐ **1.6** Liste três recursos que são T-SQL e não SQL padrão (ex.: **TOP**, **IDENTITY**, **GETDATE()**) e o equivalente em SQL padrão, quando existir.

Módulo 2 — Ambiente

Módulo de configuração, por isso tem menos exercícios. O objetivo é ter um ambiente estável para o resto do curso.

- ☐ **2.1** Suba o SQL Server via Docker **com volume persistente**. Crie um banco qualquer, remova o container, suba novamente e confirme que o banco continua existindo.
- ☐ **2.2** Conecte-se pelo SSMS. Como segundo cliente, use o VS Code com a extensão **mssql** (o Azure Data Studio foi descontinuado pela Microsoft em fevereiro de 2026).
- ☐ **2.3** Crie o repositório **sql-estudos** com a estrutura sugerida no início deste documento e faça o primeiro commit.

Módulo 3 — Hands-On SQL Básico

A partir daqui, tudo é script. O resultado deste módulo são os arquivos `00-setup.sql` e `01-carga-inicial.sql`, que você deve conseguir rodar do zero a qualquer momento.

- ☐ **3.1** Escreva o `CREATE DATABASE` de forma idempotente: o script só cria o banco se ele não existir (dica: `DB_ID('DistribuidoraDB')`).
- ☐ **3.2** Crie todas as tabelas com PK, FK, `NOT NULL` e `IDENTITY`, usando os tipos definidos no exercício 1.3.
- ☐ **3.3** Popule o banco com 5 categorias, 30 produtos, 20 clientes, 4 vendedores, 50 pedidos e cerca de 150 itens. Inclua de propósito: pelo menos 3 produtos nunca vendidos, 3 clientes sem pedido e 5 pedidos sem vendedor. Esses casos serão usados nos módulos de JOIN.
- ☐ **3.4** Tente violar a PK, uma FK e um `NOT NULL`. Anote no caderno o número e a mensagem de cada erro.
- ☐ **3.5** Aumente em 10% o preço dos produtos de uma categoria. Rode um `SELECT` antes e depois para conferir.
- ☐ **3.6** Altere para 'Cancelado' o status dos pedidos com mais de 180 dias.
- ☐ **3.7** Tente excluir uma categoria que possui produtos. Explique o erro e resolva excluindo os registros na ordem correta.
- ☐ **3.8** Crie a tabela `ProdutosBackup` com `SELECT ... INTO`. Nela, compare `DELETE` com `TRUNCATE`: o que acontece com o `IDENTITY` em cada caso? Quando cada um é permitido?

Módulo 4 — Base de conhecimentos

Módulo conceitual com exercícios curtos. As constraints criadas aqui ficam no banco para os próximos módulos.

- ☐ **4.1** Consulte `sys.indexes` e confirme que a PK de cada tabela criou um índice clusterizado automaticamente.
- ☐ **4.2** Crie um índice `UNIQUE` não clusterizado em `Clientes(Email)`.
- ☐ **4.3** Adicione as constraints: `CHECK (Preco > 0)`, `CHECK (Quantidade > 0)`, `DEFAULT GETDATE()` em `DataCadastro` e um `CHECK` que aceite apenas os status válidos. Teste violando cada uma e anote os erros.
- ☐ **4.4** No caderno, monte uma tabela comparando View, Stored Procedure e Function: o que cada uma retorna, se pode alterar dados, se pode ser usada dentro de um `SELECT` e se aceita parâmetros.
- ☐ **4.5** Abra 3 procedures reais do projeto da empresa (somente leitura, sem copiar código para fora). Para cada uma, identifique: parâmetros, tabelas usadas, se usa transação e se trata erros.

Módulo 5 — Cláusulas e Operadores

Módulo mais importante do curso. Estas consultas aparecem dentro de praticamente toda procedure.

- ☐ **5.1** Liste os produtos com preço entre dois valores, ordenados por preço decrescente e, em caso de empate, por nome.

- ☐ **5.2** Liste as cidades distintas dos clientes. Depois, conte quantas UFs diferentes existem usando `COUNT(DISTINCT)`.

- ☐ **5.3** Mostre os 5 produtos mais caros. Repita com `TOP 5 WITH TIES` e explique a diferença no resultado.

- ☐ **5.4** Implemente paginação com `OFFSET/FETCH` usando as variáveis `@Pagina` e `@TamanhoPagina`. Escreva no caderno a fórmula do `OFFSET`.

- ☐ **5.5** Com `LIKE`, encontre clientes cujo nome: começa com 'M'; contém 'silva'; tem email terminado em '@gmail.com'; tem 'a' como segunda letra.

- ☐ **5.6** Escreva uma consulta com `AND/OR` que retorna resultado errado sem parênteses. Corrija e explique a precedência entre os operadores.

- ☐ **5.7** Liste os pedidos sem vendedor. Explique por que `WHERE VendedorId = NULL` não retorna nada.

- ☐ **5.8** Mostre o maior e o menor preço de cada categoria.

- ☐ **5.9** Em `Pedidos.VendedorId`, compare `COUNT(*)`, `COUNT(VendedorId)` e `COUNT(DISTINCT VendedorId)`. Explique por que os três resultados são diferentes.

- ☐ **5.10** Calcule o valor total de cada pedido com `SUM(Quantidade * PrecoUnitario)`.

- ☐ **5.11** Conte os pedidos por status e por mês/ano.

- ☐ **5.12** Com `HAVING`, liste os clientes com mais de 3 pedidos e as categorias com preço médio acima de um valor. No caderno, escreva a ordem lógica de execução: `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`.

- ☐ **5.13** Use `IN` com uma lista fixa e depois com uma subconsulta. Em seguida, reproduza a armadilha: `NOT IN` com uma subconsulta que retorna `NULL`. Explique o resultado.

- ☐ **5.14** Filtre os pedidos de um mês com `BETWEEN` numa coluna `DATETIME`. Descubra por que pedidos do último dia podem sumir e corrija usando `>= início AND < início do mês seguinte`.

Módulo 6 — Joins

Os casos de borda criados no exercício 3.3 (produtos sem venda, clientes sem pedido, pedidos sem vendedor) são o que torna estes exercícios úteis.

- ☐ **6.1** Liste os pedidos com nome do cliente e do vendedor, primeiro com **INNER JOIN** e depois com **LEFT JOIN**. Explique quais pedidos desaparecem no **INNER** e por quê.
- ☐ **6.2** Liste os clientes que nunca fizeram pedido, usando **LEFT JOIN ... WHERE ... IS NULL**.
- ☐ **6.3** Liste os produtos nunca vendidos.
- ☐ **6.4** Monte um relatório com pedido, data, cliente, produto, categoria, quantidade e subtotal (5 tabelas).
- ☐ **6.5** Calcule o faturamento por categoria e por vendedor, incluindo vendedores sem nenhuma venda (valor 0, não **NULL**).
- ☐ **6.6** Escreva uma consulta com **RIGHT JOIN** e reescreva-a como **LEFT JOIN**.
- ☐ **6.7** Com **FULL JOIN** entre Produtos e ProdutosBackup, mostre o que existe só em uma das tabelas e o que mudou de preço.
- ☐ **6.8** Monte uma lista de contatos (clientes e vendedores) com uma coluna Tipo, usando **UNION** e depois **UNION ALL**. Compare as contagens e explique a diferença.
- ☐ **6.9** Armadilha clássica: num **LEFT JOIN**, coloque um filtro da tabela da direita no **WHERE** e depois no **ON**. Compare os resultados e explique.
- ☐ **6.10** Liste os 3 clientes com maior faturamento total, com nome e valor.

Módulo 7 — Transações

Base para escrever procedures seguras. Para os exercícios 7.5 e 7.6, use duas janelas de consulta no SSMS.

- ☐ **7.1** Crie um pedido completo dentro de uma transação: cabeçalho, itens e baixa de estoque, finalizando com **COMMIT**.
- ☐ **7.2** Repita o exercício anterior forçando um erro no meio (ex.: produto inexistente). Faça **ROLLBACK** e prove com **SELECTs** que nada foi gravado.
- ☐ **7.3** Envolve a transação em **BEGIN TRY / BEGIN CATCH**, relançando o erro com **THROW**. No **CATCH**, verifique **@@TRANCOUNT** e **XACT_STATE()**.
- ☐ **7.4** Com **SAVE TRANSACTION**: insira 3 itens válidos, crie um savepoint, insira 1 item inválido, volte ao savepoint e faça **COMMIT** apenas dos 3 primeiros.
- ☐ **7.5** Na janela 1, execute um **UPDATE** dentro de uma transação sem **COMMIT**. Na janela 2, tente ler a mesma linha. Observe o bloqueio e anote o que aconteceu.
- ☐ **7.6** Compare o comportamento de um erro dentro da transação com e sem **SET XACT_ABORT ON**.

Módulo 8 — T-SQL

Foco principal do estudo. As procedures deste módulo seguem o padrão usado em projetos reais: validação, transação, tratamento de erro e retorno claro.

Funções nativas

- **8.1** Datas: calcule há quantos dias cada pedido foi feito (**DATEDIFF**), o último dia do mês do pedido (**EOMONTH**) e a data de vencimento com 30 dias (**DATEADD**).
- **8.2** Textos: extraia o domínio do email com **SUBSTRING** e **CHARINDEX**. Use também **CONCAT**, **TRIM**, **UPPER** e **LEN**.
- **8.3** Compare **ISNULL** com **COALESCE**, e **CAST/CONVERT** com **TRY_CAST** ao converter um texto inválido.
- **8.4** Com **CASE**, classifique os produtos em faixas de preço (barato, médio, caro) e conte quantos há em cada faixa.

Functions

- **8.5** Crie a função escalar **fn_TotalPedido(@PedidoId)** que retorna o valor total do pedido.
- **8.6** Crie a função inline table-valued **fn_PedidosDoCliente(@ClienteId)** e use-a com **CROSS APPLY**.

Stored Procedures

- **8.7** **sp_InserirCliente**: valida se o email já existe (erro com **THROW** e mensagem clara) e retorna o novo ID via parâmetro **OUTPUT** usando **SCOPE_IDENTITY()**.
- **8.8** **sp_AdicionarItemPedido**: valida se o pedido existe e está aberto, valida o estoque, insere o item e baixa o estoque. Tudo com transação e **TRY/CATCH**.
- **8.9** **sp_CancelarPedido**: devolve ao estoque a quantidade de todos os itens e altera o status para 'Cancelado', dentro de uma transação. Não permite cancelar um pedido já cancelado.
- **8.10** **sp_RelatorioVendas @DataInicio, @DataFim, @CategoriaId = NULL**: quando a categoria vier **NULL**, considera todas.
- **8.11** **sp_ListarProdutos @Pagina, @TamanhoPagina, @Busca = NULL**: paginação com busca opcional por nome.
- **8.12** Padronize todas as procedures: **CREATE OR ALTER**, **SET NOCOUNT ON** e um comentário de cabeçalho com objetivo, parâmetros e exemplo de chamada.

Views, sequências e alterações de estrutura

- ☐ **8.13** Crie `vw_PedidosDetalhados` e consulte-a filtrando por cliente. Tente colocar `ORDER BY` dentro da view e explique o erro.

- ☐ **8.14** Crie uma `SEQUENCE` para número de nota fiscal usada por duas tabelas diferentes. Explique por que `IDENTITY` não resolveria esse caso.

- ☐ **8.15** Adicione uma coluna `NOT NULL` com `DEFAULT` numa tabela que já possui dados.

- ☐ **8.16** Remova uma coluna que possui constraint. Vai gerar erro: descubra o nome da constraint nas views de sistema (`sys.default_constraints` ou `sys.check_constraints`), remova-a e depois remova a coluna.

- ☐ **8.17** Renomeie com `sp_rename` uma coluna usada pela view do exercício 8.13. Verifique o que quebra e anote a lição sobre dependências entre objetos.

Módulo 9 — Backup

Poucos exercícios, mas cada um simula uma situação real de recuperação.

- ☐ **9.1** Faça um backup full. Apague uma tabela "por acidente". Restaure o backup como um banco separado (`DistribuidoraDB_Restore`) e recupere os dados com `INSERT ... SELECT` entre os bancos, sem sobrescrever o original.

- ☐ **9.2** Use `RESTORE FILELISTONLY` e `WITH MOVE` no restore. No Docker, copie o arquivo `.bak` para fora do container com `docker cp`.

- ☐ **9.3** Extra: faça um backup full, altere dados, faça um backup diferencial e restaure a sequência completa.

Módulo 10 — Tips & Tricks

Antes deste módulo, gere volume. Sem muitos registros, índices e planos de execução não mostram diferença.

- ☐ **10.1** Gere massa de dados: escreva um script que crie cerca de 1 milhão de pedidos (use `CROSS JOIN` ou laço `WHILE` com inserts em lote).

- ☐ **10.2** SQL Server Profiler: capture o SQL que uma aplicação ASP.NET Core com EF Core envia ao banco e analise a consulta gerada. Anote que o Profiler está depreciado e que a alternativa atual é Extended Events.

- ☐ **10.3** NOLOCK: reproduza uma leitura suja com duas sessões. No caderno, descreva quando o `NOLOCK` é aceitável e quando é perigoso.

- ☐ **10.4** Execute uma busca por Email sem índice e depois com índice, usando `SET STATISTICS IO, TIME ON`. Compare as leituras lógicas e identifique Scan e Seek no plano de execução.

- ☐ **10.5** Compare `WHERE YEAR(DataPedido) = 2025` com a versão por intervalo de datas, e `LIKE '%x'` com `LIKE 'x%'`. Explique no caderno o conceito de sargabilidade.

- ☐ **10.6** Crie um índice com `INCLUDE` que elimine um Key Lookup visível no plano de execução.

- ☐ **10.7** Consulte a fragmentação em `sys.dm_db_index_physical_stats`, gere fragmentação com exclusões em massa e compare `REORGANIZE` com `REBUILD`.

- ☐ **10.8** Compare o custo de `COUNT(*)` na tabela grande com a contagem via `sys.dm_db_partition_stats`.

- ☐ **10.9** CTE: encontre o vendedor com maior faturamento em cada mês.

- ☐ **10.10** CTE recursiva: gere um calendário com todas as datas de um ano. Depois, adicione `CategoriaPaiId` em Categorias e liste a hierarquia completa.

Módulo 11 — Além do curso

Tópicos que o curso não cobre, mas que aparecem com frequência em procedures reais. Resolva depois de concluir os módulos anteriores.

- ☐ **11.1** Subconsultas com **EXISTS** / **NOT EXISTS**: refaça "clientes sem pedido" (6.2) usando **NOT EXISTS** e compare os planos de execução.

- ☐ **11.2** Window functions: use **ROW_NUMBER** para obter o último pedido de cada cliente, **SUM()** **OVER** para faturamento acumulado e **LAG** para comparar o faturamento mês a mês.

- ☐ **11.3** Tabelas temporárias: escreva uma procedure de relatório com **#temp** e outra com variável de tabela (**@tabela**). Anote as diferenças.

- ☐ **11.4** Upsert: crie uma tabela de staging de produtos e sincronize-a com Produtos, primeiro com **MERGE** e depois com **UPDATE** + **INSERT** separados.

- ☐ **11.5** SQL dinâmico: escreva uma busca com filtros opcionais usando **sp_executesql** com parâmetros. Mostre, com um exemplo, por que concatenar texto do usuário abre brecha para SQL injection.

- ☐ **11.6** Desafio final: chame as procedures do módulo 8 a partir de uma API ASP.NET Core usando Dapper ou ADO.NET, incluindo leitura de parâmetro **OUTPUT** e tratamento do erro gerado pelo **THROW**.